

Introduction to Digital Humanities

Group 5: Project report

Victor Close, r1042528, victor.close@student.kuleuven.be
Koji Okumura, r1071516, koji.okumura@student.kuleuven.be
Petar Soldo, r1076709, petar.soldo@student.kuleuven.be

05/01/2026

1 Brief description of the dataset and columns

The dataset used for this project was the Books divided by Genre from the Digitised 19th century books dataset. It is, as the name implies, a dataset of the digitised 19th century books collection from the British Library (the so-called Microsoft Books). The data is divided into five Excel workbooks, each corresponding to one genre (with one being the “undetermined” genre). Truly genred files are further divided into sheets by types (which are similar to sub-genres).

Each row represents a book and each column a different piece of information about the book. The columns broadly contain data about the language of the book, its author(s), title(s), edition(s) and imprint(s), type and the genre. Not all rows have information in each of the columns.

We chose to work with all five Excel files (i.e. genres), using the Personal author, Title and Imprint columns, while preserving the rest, because it was the optimal approach to get the data we needed for the individual part of the project, which were: (i) a linguistic analysis of titles (how title features connect to date and place of publishing), (ii) a match of Croatia or classics related publications, and (iii) a genre assignation to the works from the Unspecified file using machine learning based on book titles and genres from other files.

2 Workflow

Upon deciding which columns and files to clean, we divided the work by assigning every team member one column to clean. This seemed efficient, as the data in a column is structured in the same way across different Excel spreadsheets, but (naturally) not across columns. Thus, finding a way to clean the column can be applied to all the spreadsheets. This approach also allowed us to work on the columns separately, each at their own pace and then combine them once we’re finished.

The very first problem we ran into was the lack of proper dataset documentation or guidelines which we could consult for details on the content and format of the data in the columns.

We chose to clean the column Personal author, because it seems that it contains the information on the “main” author of the work. Corporate author and Other authors seem to contain organizations and people connected to the book, but not the main author(s). Further on, we cleaned the Title column, which contains the Title of the book. Lastly, imprint contains information about the place and date of the edition in the row.

3 Cleaning process

All the characters in our columns were normalized to their Unicode canonical equivalents, using the NFC method. This was done in order to ensure that less common Unicode characters and letters (like the Slavic diacritics “č”, “ć”, etc.) are captured by regular expressions and other cleaning and analysis methods.

The code used for this was:

```
Python/Jython
import unicodedata
return unicodedata.normalize("NFC", value)
```

In addition, all leading, trailing and double whitespaces were removed using OpenRefine’s premade common function.

3.1 Cleaning the *Authors* column

The goal of cleaning this column was to get separate columns for the author’s name, title or profession, birth and death years or years of activity, and pseudonym. Two or more authors were kept in the same column, and data pertaining to them in other columns was separated by a semicolon for each author.

The original column was kept and a copy of it, called *Personal_author_clean*, was created and it was used as a basis to create all other columns. The same process was applied for each step of the cleaning process: first the desired data was extracted from the original column to a new column and then the original column was cleaned from this same data.

First, all years were extracted and placed in a column *Personal_author_years*. This includes dates of birth and death in yyyy-yyyy format, years accompanied by an abbreviation for death, birth and approximative (e.g., d., ca., b.), and years of activity (i.e. floruits, marked by fl.):

```
value.find(/,\s((fl.|d.|ca.|b.|approximately)(fl.|d.|ca.|b.|approximately))[-?]+\.*$/)[0]
```

Next, pseudonyms were moved to a *Personal_author_pseudonym* column. They were identified as the values following the abbreviation “pseud.” and we used the expression `value.find(/pseud.*$/)[0]`

Then, honorary titles and occupations of the authors were extracted in a column called *Personal_author_titles*. This was the most manual part of cleaning, because it required a somewhat longer exploration of the data to gather all the possible values of titles to be separated.

```
grel:value.find(/,\s(the)*grel:value.find(/,\s(American|City|Dramatic|Historical
|(Right\s)*Hon\.|Late|Miscellaneous|Poetical|Political|Philosophical|the)\s(
Actor|Agent|Architect|Assistant|Attorney|Author|Baron|Barrister|Bart|(
successively\s)Bishop|Captain|Chief|Clerk|Colonel|Composer|Count|Curate|D\.D|
Dean|Director|Dramatist|Duke|Earl|Editor|Esq|Fellow|General|Geographer|
Historian|Journalist|Keeper|Lady|Lawyer|Lieut\.|LL\.D\.?|Lord|M\.D\.?|(Brigade
\s)Major|Marquis|Master|Member|Minister|Novelist|Poet|Prebendary|President|
Printer|Professor|Provost|Publisher|Rector|Rev\.?|Schoolmaster|Secretary|
Sergeant|Sir|Tradesman|Tragedian|Traveller|Vicar|Writer)\b.$/)[0]
```

Some titles began with ordinal numbers (e.g., 1st, 5th) and thus remained in the *Personal_author_years* column. These were later extracted and merged into *Personal_author_title* by matching digits followed by ordinal suffixes (*st*, *nd*, *rd*, *th*), while excluding cases like “cent.” to avoid picking up centuries.

```
value.find(/(?!\.*(cent|Cent))\d+(st|nd|rd|th).+/)[0]
```

Finally, multiple authors in the *Personal_author_column* were separated by a semicolon to make further analysis easier.

```
value.replace(/,*\sand\s/, "; ")
```

For each of the data extraction steps, there is an additional, similar step for cleaning the “clean” column. The code is almost always the same, but `value.replace` is used (instead of `value.find`) and the expression is replaced by an empty string (“”). After this, a small manual cleaning was performed (by faceting and filtering the columns) to catch cases not covered by the regular expression (mostly due to inconsistent notation).

Subsequently, for reconciliation purposes, Years were split into date of birth column (using `value.find(/(d.*)|-(\d1,4.*$)/)[0]`) and date of death column (using `value.find(/(b.*)|(\d1,4-)/)[0]`).

3.2 Cleaning the *Imprint* column

Two new columns were created from the Imprint one. A column Date, and a column Location. We’re not interested in other information available in the Imprint such as editor name, the location of printing, the person for whom it was printed, etc. In any case, this information remains in the original Imprint column, should we need it at some point. We also noticed that several cities could appear in the same Imprint cell, without any information about which one is the place of origin. For our analyses, we will assume it to be the first one and will hence only keep this one in the new Location column.

The first task was to create a Date column by finding all four digit numbers.

```
value.find(/[0-9]{4}/)[0]
```

The second task was to isolate a Location column. To do this, we copied the Imprint column and cleaned it until we were left with just the location. The first step in this phase was removing dates, punctuation, and small words. In the following lines, text between square brackets and parentheses was removed; digits and various noise was replaced by a whitespace (to avoid accidentally concatenating name like San-Francisco to SanFrancisco); all words not starting with a capital letter are removed (unfortunately discarding cities which were not well written); all simple letters and two-letter words were removed; city names consisting of two characters strings separated by a whitespace were reconciled into a single entity. With these, all city names should normally appear without any white space.

```

value.replace(/\[.+\\]/, "").trim()
value.replace(/\[.+\\]/, "").trim()
value.replace(/[[0-9]]|\.|\\&|\\[\\]|\\?\\:|\\;|\\(|\\)|\\-/, "-").trim()
value.replace(/\\s[a-z]+/, "") and value.replace(/^[a-z]+/, "")
value.replace(/\\s?[A-Z][a-z]?{1}?\\s/, "-")
value.replace(/\\s[A-Z][a-z]?{1}?$/ , "-")
value.replace(/New\\s/, "New—")
value.replace(/San\\s/, "San—")
value.replace(/Saint\\s/, "Saint—")

```

To isolate cities from the Location column, we split the text on whitespaces, creating 20 columns, and removed columns 7 to 20 as they contained mostly blanks or irrelevant imprint details. From column 6 onward, city names appeared more frequently, so we used a text facet to identify the most common ones and applied a regex to keep only those occurring five times or more, setting others to blank. While this introduces some bias toward well-represented cities like London or New York, retaining all would have required excessive manual work beyond the project’s scope.

For column 6 we just kept London by using `if(value == "London", value, null)`.

For columns 5-3 we used Python for a slightly longer condition, adding more cities in each column.

```

v = value
if v in ( 'London', 'New-York', 'Edinburgh' ): #in column 5
if v in ( 'London', 'Edinburgh', 'New-York', 'Cambridge', 'Dublin', 'Liverpool',
          'Nuremberg', 'York' ): #in column 4
if v in ( 'London', 'New-York', 'Edinburgh', 'Stanford', 'Leipzig', '
          Cambridge', 'Dublin', 'Toronto', 'Wellington' ): #in column 3
    return value
else:
    return None

```

[language=Python] The operation wasn't applied on Columns 2 and 1, as they count a large number of cities. We then merged the six remaining columns into a new one called *Location_col_merged*. Text facets were used to identify and uniformize 405 clusters. The new *Location_col_merged* column was split on whitespaces to have the first column represent most cities present in the original *Imprint*. We're aware that a small percentage of information is missing or incorrect in *Location_clean*. Therefore, we keep an intermediary column *Location* (renamed *Location_intermediate*) along with *Imprint*. *Location_clean* is then clustered a last time for different spelling in the city names and some light manual cleaning was added.

3.3 Cleaning the *Imprint* column

The goal was to clear everything that didn't pertain to the title itself, including the authors. A copy of the Title column named *Title_cleaned* was created. First, we remove the string "[electronic resource]" and other strings in parentheses, repeating the second step until all was removed (for nested structures).

```
value.replace("[electronic-resource]", "").trim()
value.replace(/\([^(\)]*\)\.?/, "").trim()
```

Next, we remove the strings in square brackets by making a copy column named *Title_copy*. We then apply the two following codes, first one to eliminate the brackets, second one to preserve whole titles covered with squared brackets. This is once again repeated as much time as need (because of nested structures).

```
value.replace(/\[ [^\[\]]* \] \. ? / , "").trim()
if( value == "", cells["Title_copy"].value.substring(1, value.length() - 1),
    value).trim()
```

We then removed whitespaces just before punctuations and collapsed double whitespaces.

```
value.replace(/\\textbackslash s+([.,;]+) / , "\\$1")
value.replace(/(\\textbackslash s){2,} / , "\\$1")
```

Finally, we split the strings after "By" into a different column named *Title_after_By* using the "By" + `value.trim()` expression.

In this data set, some titles seem to include the author. We assume that "By" (with capital case B) clearly indicates the author information. We do not do anything for "by" (with small case letters) since it is very difficult to separate the author information properly.

Lastly, language detection was performed with Google API using the following

```
{ "https://translation.googleapis.com/language/translate/v2/detect?key=[MY-
API-KEY]&q=" + escape(value, "url") }. \footnote{The real API key is
hidden due to security reasons.}
```

The detected language was extracted from the received JSON using:

```
value.parseJson().data.detections[0][0].language
```

Next, we transformed the two-letter language code (ISO 639-1) into the three-letter language code (ISO 639-2). Note that the Google Cloud Translation API uses the former, but the original Excel files use the latter.

First, we prepared the CSV file for lookup (*language_code_transform.csv*) and created a new project named "language_code_transform".

Then, looking at the project "language_code_transform", we transformed the two-letter language code into the three-letter language code using the following GREL expression:

```
cell.cross("language_code_transform", "two_letter_code")[0].cells["
three_letter_code"].value
```

4 Wikidata reconciliation

The columns were reconciled. *Author_clean* was reconciled using dates of birth and death and pseudonym as adjuvant columns and around 40% of names was successfully reconciled. Not much was able to be done with manual reconciliation, as opposed with the *Place* column which benefited from it greatly and had more than 90% of reconciled data. *Titles*, being the most difficult to reconcile, had 19% of reconciled cells.

5 Checking for errors

Independent test for errors were performed. Four separate random samples of rows were extracted (using Pandas for random sampling) and manually checked. Each of the samples had 100 rows, which would give us a confidence interval of 90% and a margin of error of around 8% when estimating errors in the whole dataset. Three of the samples were checked for the quality of the data cleaning process (one for each of the cleaned columns). Cleaned columns created from *Personal author* and *Imprint* had 9 errors/inconsistencies in 100 rows, while *Title* had 13. Fourth sample was checked for the correctness of the data itself (because faulty data was spotted during the cleaning) and 2 out of 100 rows had incorrect or misleading data.